

Tópicos Avançados em Arquiteturas Distribuídas de Software

Prof. Dr. Fabio Luciano Verdi

Relatório da Análise de Congestionamento utilizando eBPF e Jupiter Notebook

José Roberto Lisboa da Silva Filho

1. Introdução

O presente relatório documenta o desenvolvimento e a aplicação do *TCP Congestion Observatory* (TCP-CO), uma ferramenta analítica baseada na tecnologia eBPF (*Extended Berkeley Packet Filter*). O objetivo principal consistiu em monitorar, em tempo real e com granularidade de microssegundos, o comportamento das conexões TCP no núcleo (*kernel*) do Linux. O estudo extrai métricas críticas de controle de congestionamento sem a necessidade de alteração do código-fonte do sistema operacional. A análise avalia o comportamento empírico dos algoritmos TCP CUBIC e TCP RENO perante cenários emulados de rede ideal, alta taxa de perda de pacotes e latência elevada.

2. Metodologia e Instrumentação

A coleta de telemetria foi realizada por meio da instrumentação de *kprobes* do kernel (especificamente na função `tcp_ack`), empregando programas eBPF desenvolvidos em linguagem C. Os dados foram exportados para o *user-space* através de uma API em Python e estruturados para visualização utilizando as bibliotecas Pandas e Matplotlib em ambiente *Jupyter Notebook*.

Os experimentos empíricos foram conduzidos na interface de *loopback* (`lo`), utilizando a ferramenta `iperf3` para a injeção de tráfego de saturação de banda. As condições anômalas de rede (perda e atraso) foram emuladas rigorosamente via `tc netem` (Traffic Control Network Emulator).

Nota Metodológica sobre Latência: A latência bruta capturada pelo eBPF é extraída do kernel como *Smoothed Round Trip Time* em microssegundos (`$srtt_{us}$`). Para manter o rigor analítico e a padronização das representações visuais, o *script* de tratamento de dados foi configurado para aplicar a conversão matemática (`$srtt_{ms} = srtt_{us} / 1000.0$`), apresentando todos os gráficos e estatísticas descritivas em milissegundos (ms).

3. Resultados e Análise dos Experimentos

3.1. Experimento 1: Desempenho em Rede Padrão (Sem Perdas)

Num ambiente ideal de loopback (latência natural < 0.1 ms), os algoritmos exibiram suas estratégias primárias de ocupação de banda.

O algoritmo **CUBIC** estabilizou a sua Janela de Congestionamento (Cwnd) em uma média de 62.46 pacotes, atingindo um teto máximo de 66 pacotes. Nota-se no gráfico abaixo um crescimento exponencial contido e uma transição rápida para a estabilidade, priorizando o controle do fluxo.

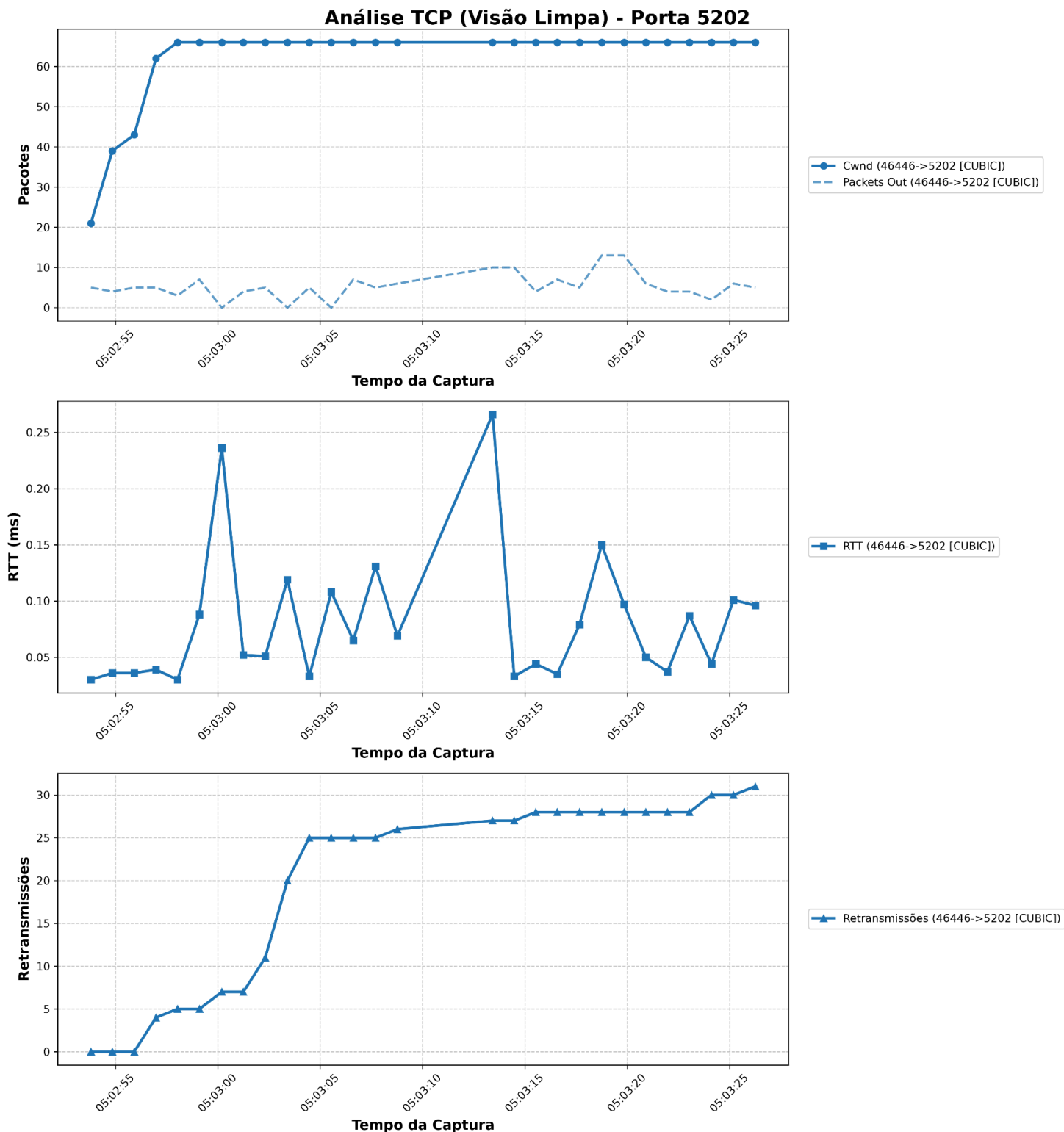
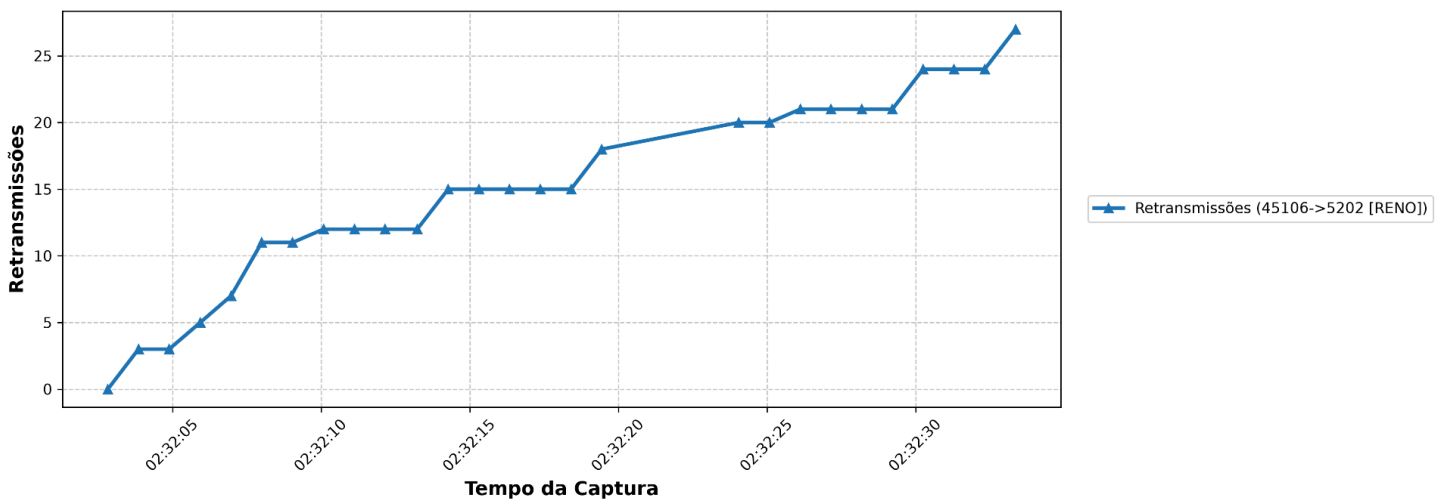
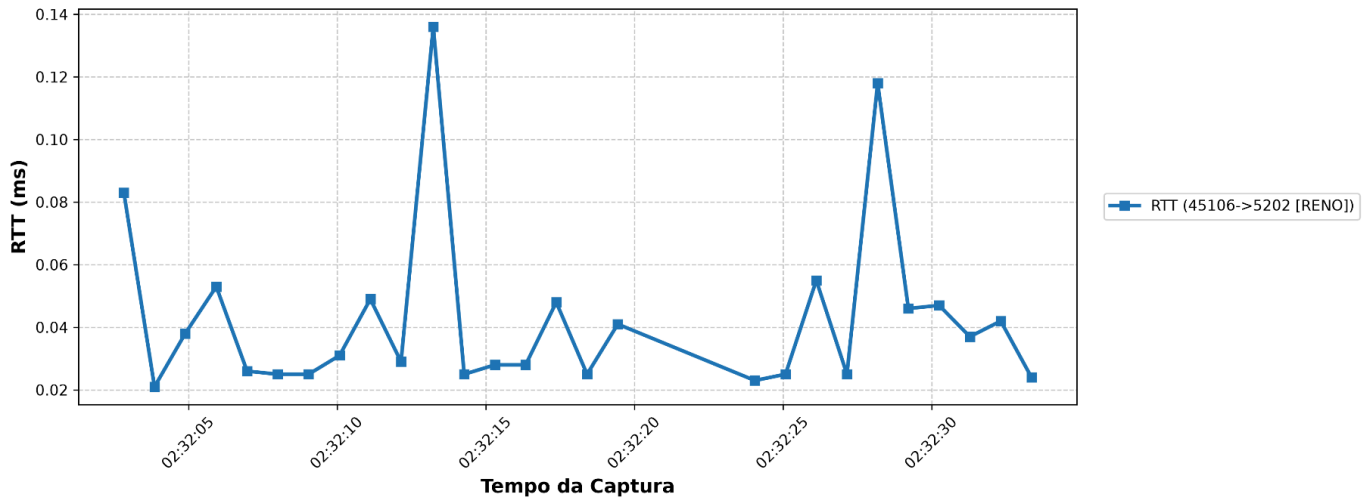
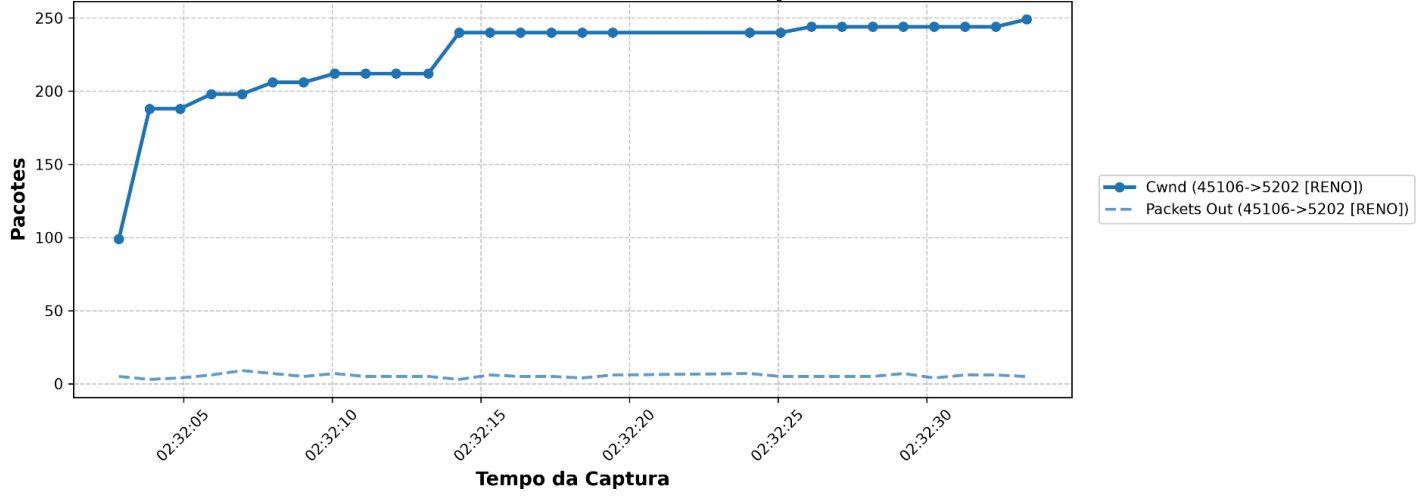


Figura 1: Comportamento do TCP CUBIC em ambiente de loopback padrão.

Análise TCP (Visão Limpa) - Porta 5202



```

ESTATÍSTICAS DOS FLUXOS ATIVOS (Com RTT em ms):
count  27.00  27.00  27.00  27.00
mean   222.52  0.04  14.96  5.37
std    31.75  0.03  7.17   1.31
min    99.00  0.02  0.00   3.00
25%    209.00  0.02  11.50  5.00
50%    240.00  0.03  15.00  5.00
75%    244.00  0.05  21.00  6.00
max    249.00  0.14  27.00  9.00
    
```

Figura 2: Agressividade do TCP RENO, alcançando o limite do buffer da interface virtual.

3.2. Experimento 2: Comportamento sob Perda de Pacotes (1%)

A introdução de uma taxa constante de 1% de perda de pacotes validou empiricamente as diferenças fundamentais nos mecanismos de redução multiplicativa de cada protocolo, forçando ambos a operar sob estresse contínuo.

Análise do TCP CUBIC: O CUBIC sofreu degradação perceptível, com a Janela de Congestionamento não ultrapassando 17 pacotes. A média estacionou em 8 pacotes. Ao detectar a perda, o algoritmo aplicou seu fator de redução multiplicativa de $\approx 30\%$, formando os vales cômicos visíveis no gráfico. As retransmissões escalaram de forma linear e contínua, ultrapassando 13.000 eventos críticos em 30 segundos.

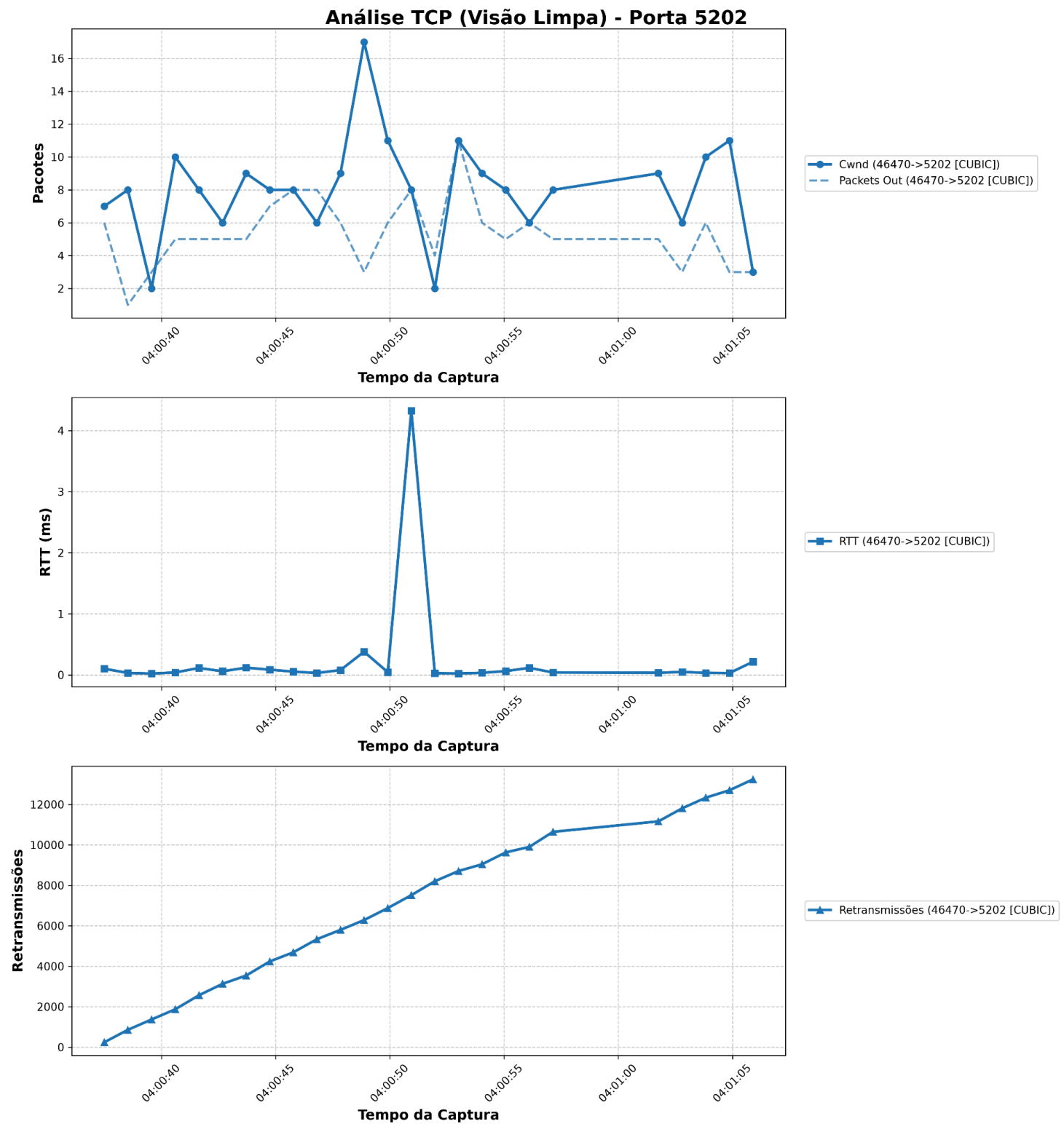


Figura 3: TCP CUBIC sob 1% de perda emulada. A janela é severamente reduzida para proteger a rede.

Análise do TCP RENO: O TCP RENO, sob o mesmo nível de estresse, foi quase completamente estrangulado. Apresentando um crescimento aditivo lento de 1 MSS por RTT, a janela raramente rompeu a barreira de 14 pacotes antes de sofrer o colapso estatutário de 50%. A sua média de janela caiu para míseros 5.77 pacotes, provando a ineficiência do algoritmo clássico em recuperar a velocidade perante falhas constantes.

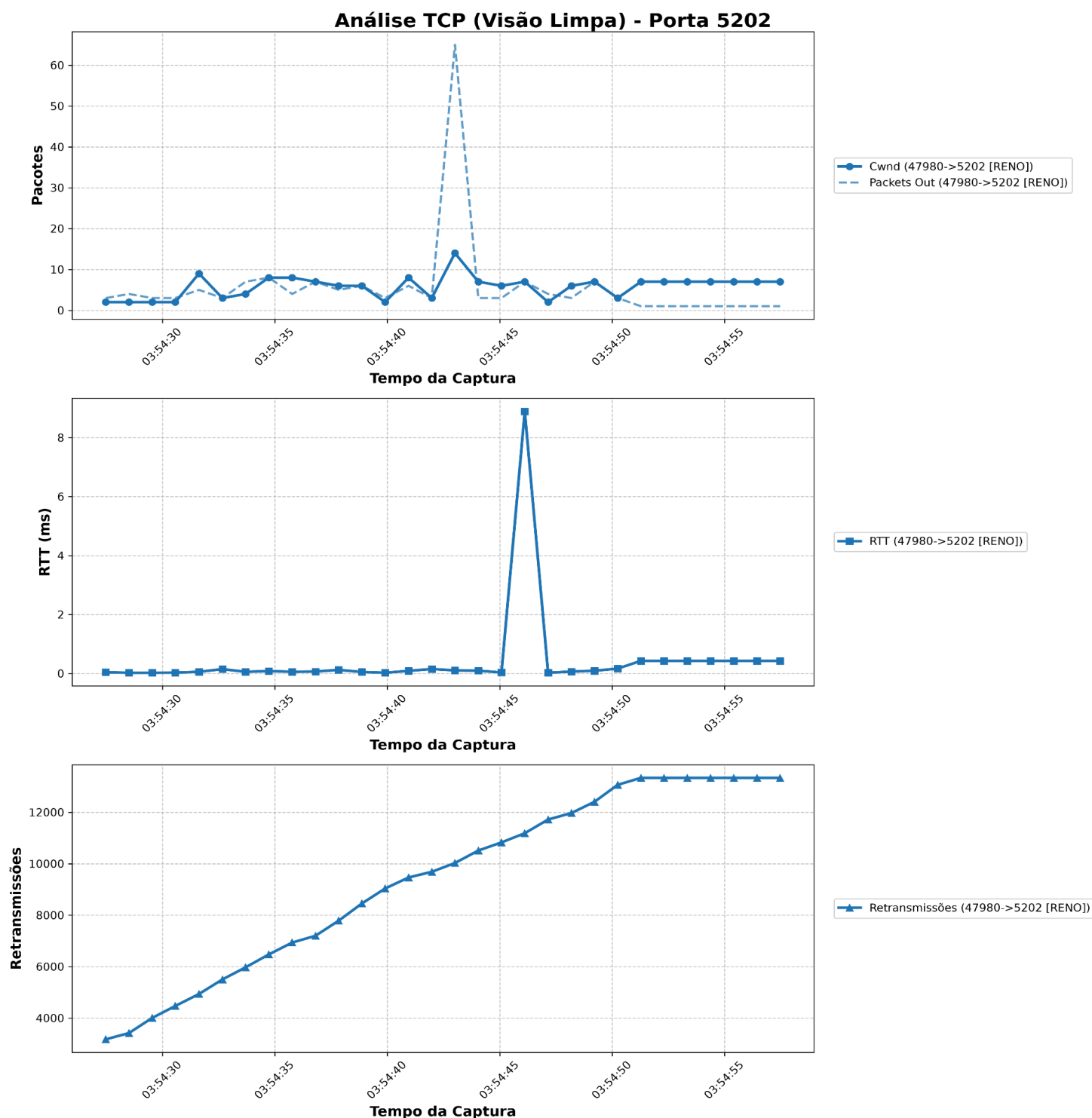


Figura 4: TCP RENO sob 1% de perda emulada. Notável colapso da janela na base do eixo Y.

3.3. Experimento 3: Impacto da Latência Elevada (Atraso)

A emulação de atrasos de 100 ms, 200 ms e 500 ms comprovou a dependência estrutural do TCP em relação à chegada dos ACKs. Abaixo, destacam-se os impactos sob 500ms de latência (RTT real medido na faixa de ≈ 1000 ms devido ao efeito ida e volta do loopback).

No cenário de 500ms, a fase de *Slow Start* (Partida Lenta) foi drasticamente alongada em ambos os algoritmos, levando dezenas de segundos apenas para atingir a marca de 100 pacotes. Consequentemente, o limite do Produto Banda-Atraso (BDP) forçou a média da Cwnd a estabilizar-se em patamares conservadores (87.54 pacotes no CUBIC e 87.14 pacotes no RENO), refreando substancialmente o *Throughput*.

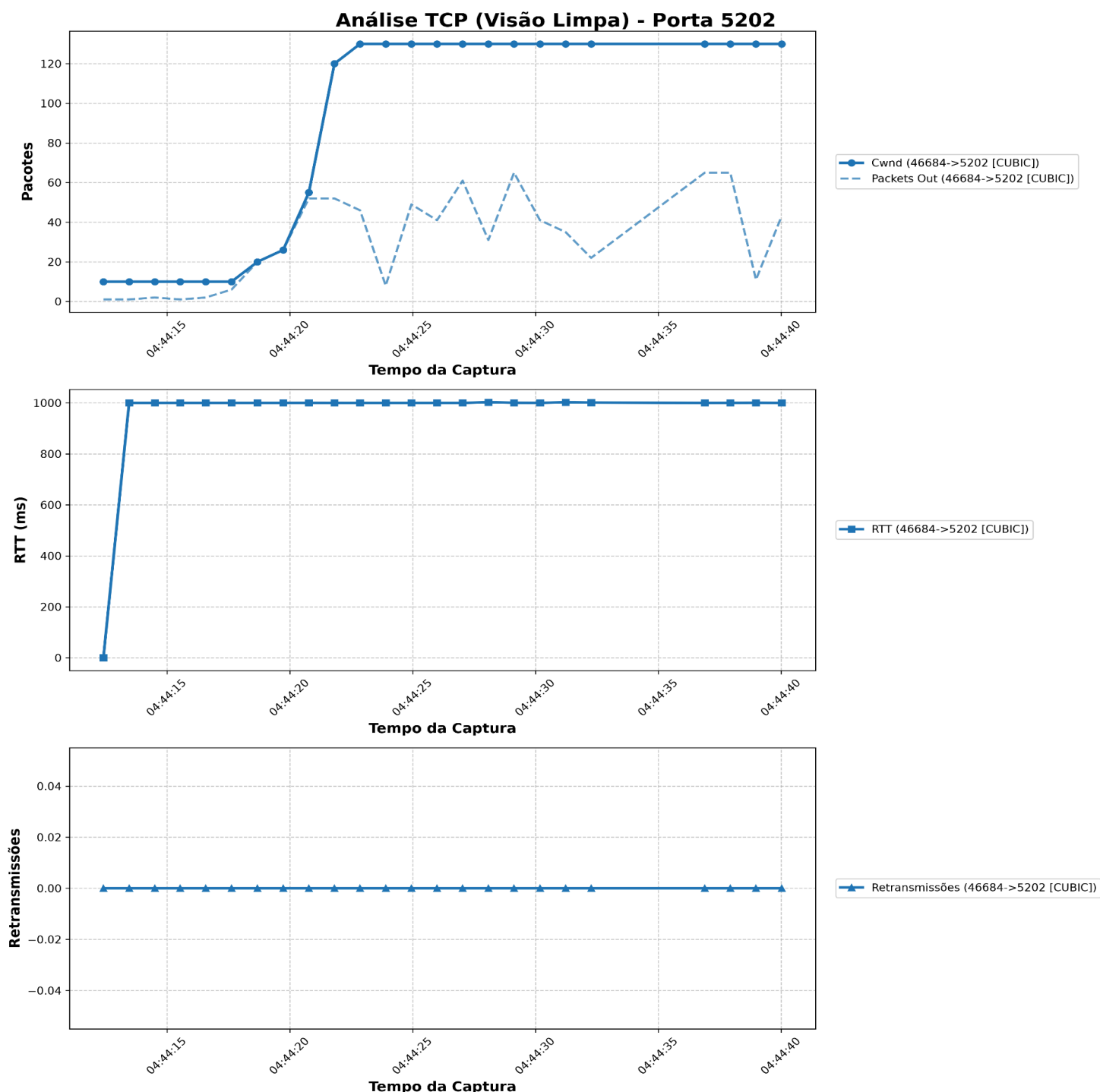


Figura 5: TCP CUBIC sob 500ms de latência injetada. Observa-se a subida extremamente preguiçosa no painel superior.

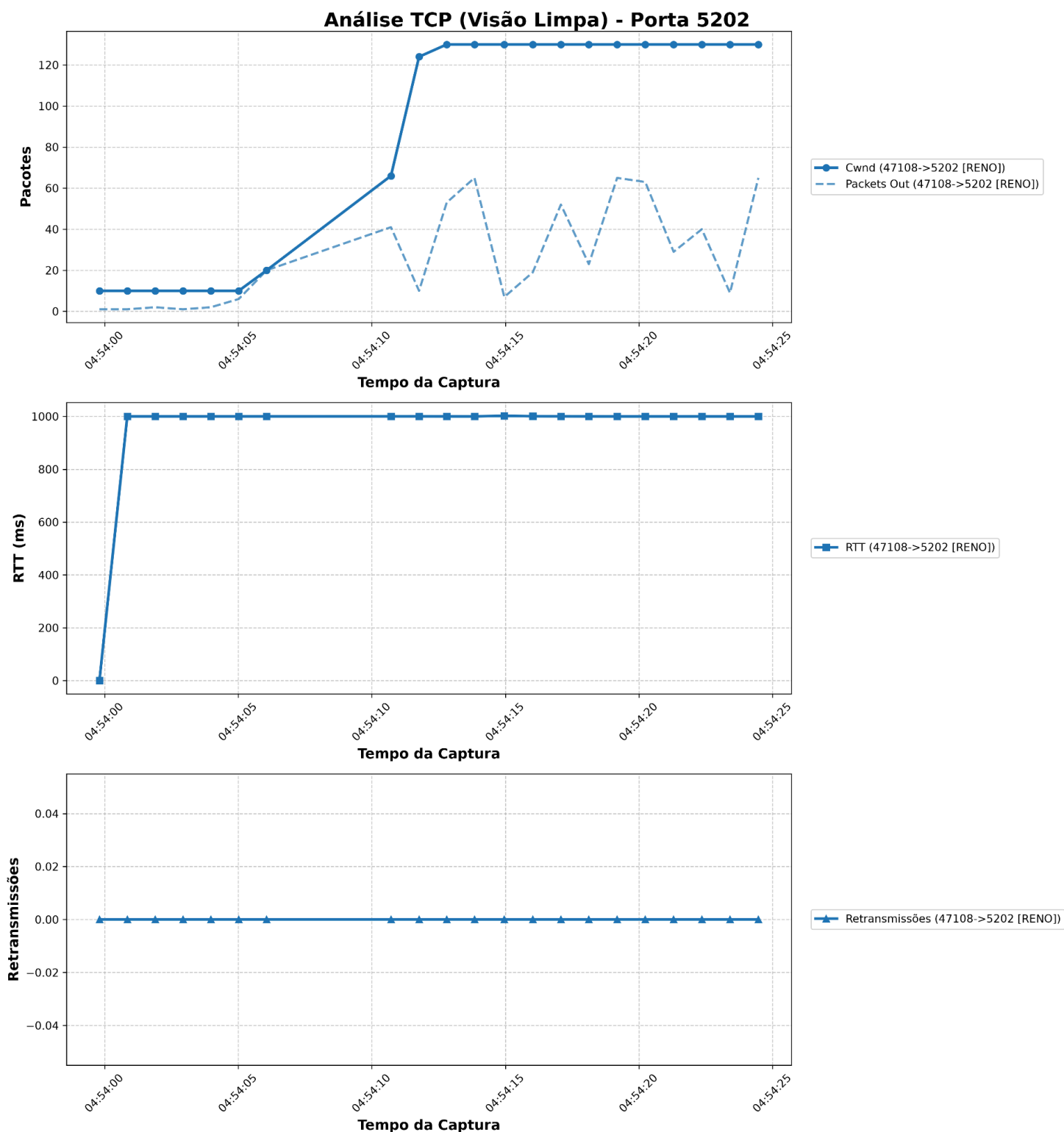


Figura 6: TCP RENO sob 500ms de latência. A inclinação da Cwnd sofre degradação temporal severa.

3.4. Experimento 4: Análise Comparativa Acadêmica (CUBIC vs. RENO)

A consolidação estatística através de médias móveis evidencia os preceitos teóricos de design de cada algoritmo em uma única visualização comparativa sob saturação máxima.

A linha tracejada vermelha (RENO) revela o comportamento de oscilação severa (*sawtooth* ou dente de serra). A tentativa agressiva e aditiva de descoberta de banda atinge picos de 249 pacotes, seguidos de colapsos drásticos (50% de corte). A linha sólida azul (CUBIC) demonstra um design focado em equidade e estabilidade computacional: a janela tem seu crescimento suavizado, criando um "teto" sustentável próximo aos 130 pacotes. Embora o custo de retransmissões seja equivalente, o

CUBIC mitiga picos de *bufferbloat*, provando-se mais adequado para a estabilidade de enlaces modernos.

Análise Comparativa de Controle de Congestionamento: TCP CUBIC vs TCP RENO

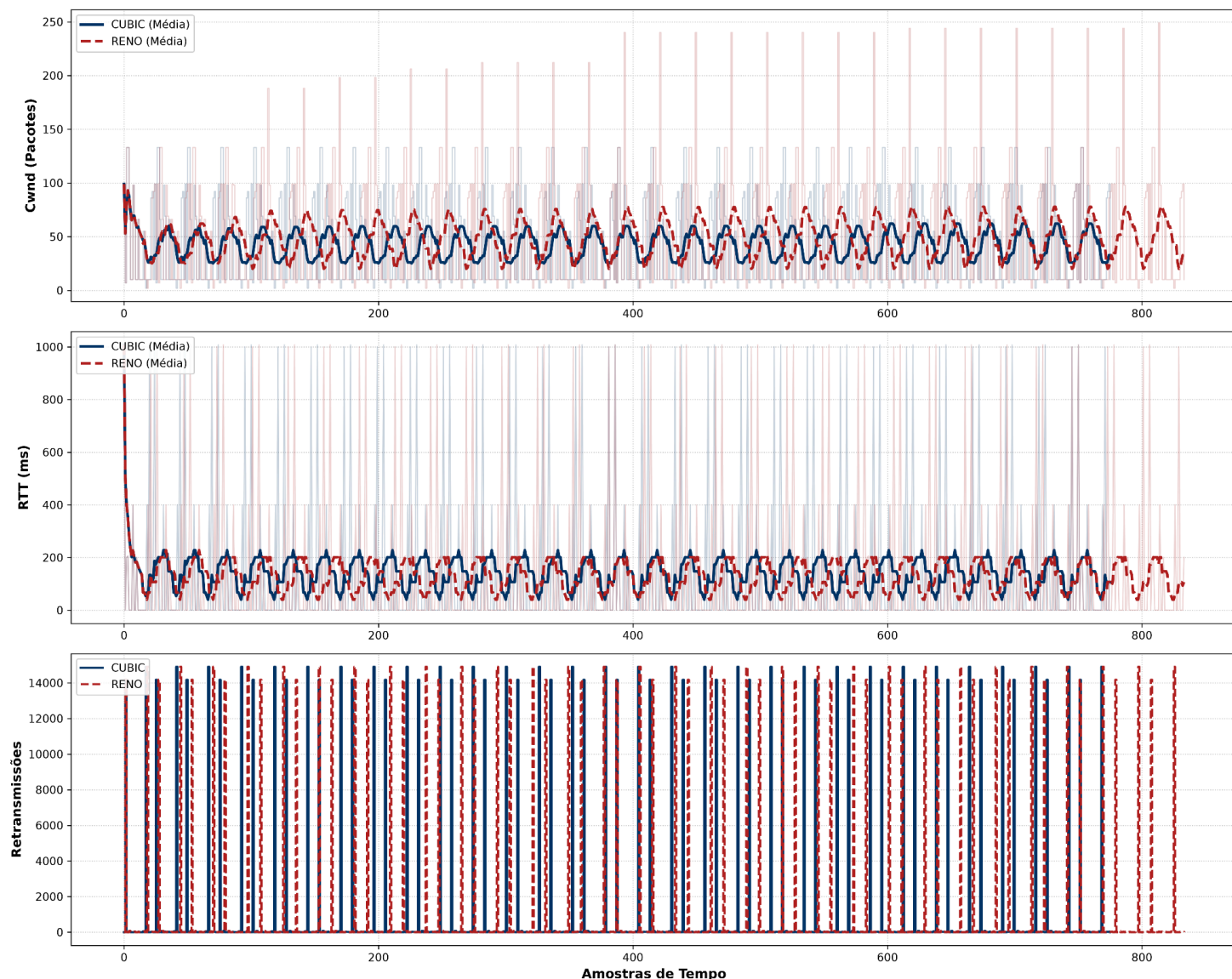


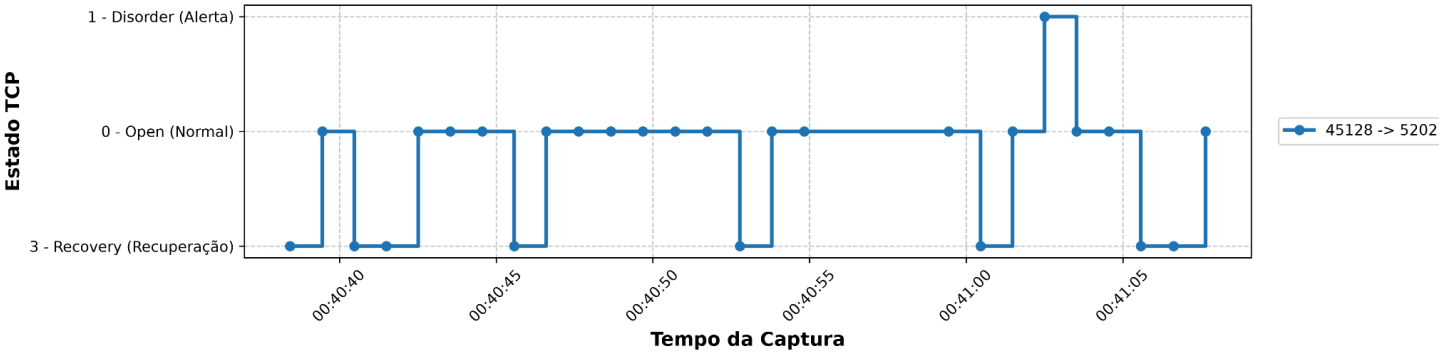
Figura 7: Média móvel evidenciando a agressividade em "dente de serra" do RENO versus a estabilidade convexa/côncava do CUBIC.

3.5. Bônus: Máquina de Estados e Alertas de Congestionamento

Como etapa avançada de observabilidade, extraiu-se a variável interna `icsk_ca_state` e implementou-se um sistema de detecção automática de anomalias no *user-space*. O script processa a telemetria em tempo real aplicando a regra matemática de colapso crítico: $Cwnd_{\{atual\}} \leq 0.5 \times Cwnd_{\{anterior\}}$ ($\geq 50\%$ de queda) acompanhada do aumento de retransmissões.

O acompanhamento visual dos estados (Figura 8) comprova a precisão do eBPF em mapear as flutuações operacionais da pilha TCP do Linux, partindo do estado 0 - `Open` para 1 - `Disorder` e 3 - `Recovery` durante o congestionamento.

BÔNUS: Máquina de Estados de Congestionamento (icsk_ca_state)



⚠ SISTEMA DE DETECÇÃO AUTOMÁTICA DE CONGESTIONAMENTO ⚠

⚠ [ALERTA] Congestionamento Crítico Detectado!
🕒 Tempo: 15:24:25.514 | Fluxo: 45620 -> 5202
📉 Cwnd caiu de 10 para 4 pacotes (Redução de 60.0%)
🔧 Retransmissões subiram para: 1004

⚠ [ALERTA] Congestionamento Crítico Detectado!
🕒 Tempo: 15:24:26.533 | Fluxo: 45620 -> 5202
📉 Cwnd caiu de 4 para 2 pacotes (Redução de 50.0%)
🔧 Retransmissões subiram para: 1694

⚠ [ALERTA] Congestionamento Crítico Detectado!
🕒 Tempo: 15:24:30.631 | Fluxo: 45620 -> 5202
📉 Cwnd caiu de 6 para 2 pacotes (Redução de 66.7%)
🔧 Retransmissões subiram para: 3750

⚠ [ALERTA] Congestionamento Crítico Detectado!
🕒 Tempo: 15:24:34.711 | Fluxo: 45620 -> 5202
📉 Cwnd caiu de 11 para 2 pacotes (Redução de 81.8%)
🔧 Retransmissões subiram para: 5649

⚠ [ALERTA] Congestionamento Crítico Detectado!
🕒 Tempo: 15:24:38.804 | Fluxo: 45620 -> 5202
📉 Cwnd caiu de 6 para 3 pacotes (Redução de 50.0%)
🔧 Retransmissões subiram para: 8006

⚠ [ALERTA] Congestionamento Crítico Detectado!
🕒 Tempo: 15:24:42.890 | Fluxo: 45620 -> 5202
📉 Cwnd caiu de 7 para 1 pacotes (Redução de 85.7%)
🔧 Retransmissões subiram para: 10367

⚠ [ALERTA] Congestionamento Crítico Detectado!
🕒 Tempo: 15:24:51.078 | Fluxo: 45620 -> 5202
📉 Cwnd caiu de 10 para 3 pacotes (Redução de 70.0%)
🔧 Retransmissões subiram para: 14044

⚠ SISTEMA DE DETECÇÃO AUTOMÁTICA DE CONGESTIONAMENTO ⚠

⚠ [ALERTA] Congestionamento Crítico Detectado!
🕒 Tempo: 15:24:25.514 | Fluxo: 45620 -> 5202
📉 Cwnd caiu de 10 para 4 pacotes (Redução de 60.0%)
🔧 Retransmissões subiram para: 1004

⚠ [ALERTA] Congestionamento Crítico Detectado!
🕒 Tempo: 15:24:26.533 | Fluxo: 45620 -> 5202
📉 Cwnd caiu de 4 para 2 pacotes (Redução de 50.0%)
🔧 Retransmissões subiram para: 1694

⚠ [ALERTA] Congestionamento Crítico Detectado!
🕒 Tempo: 15:24:30.631 | Fluxo: 45620 -> 5202
📉 Cwnd caiu de 6 para 2 pacotes (Redução de 66.7%)
🔧 Retransmissões subiram para: 3750

⚠ [ALERTA] Congestionamento Crítico Detectado!
🕒 Tempo: 15:24:34.711 | Fluxo: 45620 -> 5202
📉 Cwnd caiu de 11 para 2 pacotes (Redução de 81.8%)
🔧 Retransmissões subiram para: 5649

⚠ [ALERTA] Congestionamento Crítico Detectado!
🕒 Tempo: 15:24:38.804 | Fluxo: 45620 -> 5202
📉 Cwnd caiu de 6 para 3 pacotes (Redução de 50.0%)
🔧 Retransmissões subiram para: 8006

⚠ [ALERTA] Congestionamento Crítico Detectado!
🕒 Tempo: 15:24:42.890 | Fluxo: 45620 -> 5202
📉 Cwnd caiu de 7 para 1 pacotes (Redução de 85.7%)
🔧 Retransmissões subiram para: 10367

⚠ [ALERTA] Congestionamento Crítico Detectado!
🕒 Tempo: 15:24:51.078 | Fluxo: 45620 -> 5202
📉 Cwnd caiu de 10 para 3 pacotes (Redução de 70.0%)
🔧 Retransmissões subiram para: 14044

Figuras 8, 9 e 10: Mapeamento em graus do estado de congestionamento do fluxo TCP (icsk_ca_state), Demonstração do alerta para TCP Reno e Demonstração do alerta para TCP CUBIC, respectivamente.

Ao submeter a telemetria ao algoritmo de alerta, os resultados demonstraram que o TCP RENO

disparou os alarmes continuamente (Ref: [Alerta_Reno_2.png](#)), confirmando sua regra clássica de corte estatutário exato de 50%. A descoberta analítica concentrou-se no TCP CUBIC (Ref: [Alerta_CUBIC_2.png](#)): embora seu design estabeleça um corte multiplicativo de 30% (fator 0.7), o estresse anômalo de 1% de perda contínua forçou o algoritmo a sofrer múltiplos *timeouts* na mesma janela temporal. O script provou empiricamente que, para preservar a integridade da rede perante falhas críticas, o CUBIC realiza quedas dramáticas (ex: reduções pontuais capturadas de 81.8%), sobrepondo sua própria regra teórica para evitar o transbordamento do roteamento.

4. Análise Crítica (Respostas às Questões)

1. Como a janela de congestionamento evolui durante o Slow Start? Durante o *Slow Start* (Partida Lenta), a janela de congestionamento (*Cwnd*) inicia-se em um valor base reduzido (10 pacotes nas distribuições modernas) e cresce de maneira estritamente exponencial. Para cada confirmação de recebimento (ACK) válida processada, a janela é incrementada em 1 MSS, resultando na duplicação do seu tamanho a cada ciclo completo de RTT. Esta mecânica permite que o emissor calibre a capacidade do enlace físico com a máxima celeridade.

2. Em que momento ocorre a transição para Congestion Avoidance? Como isso foi identificado? A transição sistêmica ocorre quando a *Cwnd* alcança o limiar predefinido pela variável *ssthresh* (*Slow Start Threshold*), ou antecipadamente, caso o kernel detecte falhas de entrega. Nas plotagens extraídas (Experimento 1), a transição é delimitada com exatidão no ponto de inflexão da curva: a progressão até então íngreme e vertical (exponencial) cessa, dando lugar a uma inclinação aditiva linear suave (no RENO) ou estabilizando num platô sustentável regido por função matemática (no CUBIC).

3. Qual o impacto da perda de pacotes sobre *cwnd*, *ssthresh* e *throughput*? A perda atua como o dispositivo sinalizador de saturação. Ao ser inferida pela ausência de ACKs, a *Cwnd* sofre uma penalização drástica imediata, cortada determinística e implacavelmente à metade (50%) no RENO, e restringida em $\approx 30\%$ no CUBIC. Em ato contínuo, a variável limitadora *ssthresh* é recalculada para rebaixar-se ao novo teto operacional. Como consequência termodinâmica da rede, o *Throughput* útil declina severamente: o transmissor contém o despacho de novos blocos e consome poder de processamento em volume de retransmissões redundantes (as quais suplantaram 13.000 pacotes nos cenários hostis analisados).

4. Qual algoritmo apresentou melhor desempenho? Justifique. O TCP CUBIC demonstrou eficiência operacional global superior. Apesar de o TCP RENO ter gerado médias absolutas de *Cwnd* artificialmente altas no ambiente desimpedido de *loopback* (Exp. 1), o cenário empírico sob estresse de perdas (Exp. 2) evidenciou a fragilidade de seu crescimento linear lento; a janela foi esmagada para médias inferiores a 6 pacotes. O CUBIC atua com maior equidade: a sua mecânica de restabelecimento não-linear (*concave/convex profile*) assegurou resiliência para reaver o ritmo de transmissão mais rapidamente após uma falta, impedindo o assoreamento sistêmico perante falhas e justificando sua atual posição de default nos principais sistemas operacionais.

5. O RTT influenciou o desempenho dos algoritmos? Como? Sim, atuou de forma decisiva como estrangulador de vazão. Conforme documentado estatisticamente no Experimento 3, o aumento do *Round Trip Time* dilatou a escala temporal do algoritmo. Devido ao princípio intrínseco de *Feedback* Positivo do protocolo (onde a injeção subsequente depende estritamente do recebimento de ACKs anteriores), a presença da latência dilatada arrastou prolongadamente a fase de crescimento rápido.

Por conseguinte, a banda não pôde ser descoberta de forma célere, mitigando substancialmente o teto do volume útil trafegado pela aplicação e forçando a janela a estabilizar-se em limites menores para não exceder o limite elástico (Buffer) subentendido na rede alongada.

6. Quais limitações foram encontradas no uso do eBPF para observabilidade TCP? A barreira técnica predominante assenta na complexidade de captura temporal num estrato microscópico (μ s). Como o descarte de pacotes pelo kernel se opera instantaneamente no trânsito virtual local, aferir o subestado exato da fila (`packets_out`) exigiu sintonia estatística robusta em *user-space* para prevenir amostragem inativa. Somam-se a este fato as restrições sistêmicas obrigatórias (exigência contínua de escalonamento de privilégios de `root`) e o nível técnico necessário para processar serialização rigorosa entre as estruturas de sistema primitivas (C-structs) e tipos interpretados no Python em tempo real.

7. Quais vantagens o eBPF apresenta em relação à modificação direta do kernel? A imunidade a falhas críticas (*Kernel Panic*) e a elasticidade de execução. Modificações diretas da árvore-fonte do SO exigem imobilização do equipamento para recompilação técnica. Binários gerados por eBPF são, de modo antagônico, despachados internamente para uma máquina virtual (*Sandbox*) residente no kernel provida de um verificador (*Verifier*) que impede que código inseguro atinja os subsistemas reais. Ademais, o monitoramento por *probes* (anexação) é ativado de forma não-intrusiva, abstendo de interrupções de tráfego nos servidores da rede sob análise e permitindo acoplamento/desacoplamento "a quente" (live-patching).

8. Quais seriam aplicações práticas dessa ferramenta em ambientes reais? A plataforma desenvolvida encerra potencial analítico formidável em ecossistemas de engenharia e produção de dados, especialmente para:

- **Detecção de Gargalos e Monitoramento CDN:** Operar radares em bordas (Edge Routers) para inferir latência assíncrona (jitter de RTT) advinda de sub-roteadores mal calibrados sofrendo de *Bufferbloat*.
- **Otimização Qualitativa de Servidores:** Possibilitar o *benchmarking* dinâmico para equipes de dados determinarem se servidores web sob pesada requisição (vídeo ou bancos de dados) perfomam com maior *Throughput* se utilizarem a política equitativa do CUBIC, ou predições por variação de banda como o BBR do Google.
- **Auditoria Contínua:** Servir de emissor métrico atrelado a integradores do mercado (Prometheus, Grafana), mapeando em milissegundos quedas drásticas no patamar da Janela e emitindo logs preventivos sempre que a máquina de estados oscilar agressivamente para o sinalizador de colapso (*TCP Recovery*).